

ASSIGNMENT - 7.5

2303A51042

Batch : 29

Task 1 (Mutable Default Argument – Function Bug)

Task: Analyze given code where a mutable default argument causes unexpected behavior. Use AI to fix it.

Bug: Mutable default argument

```
def add_item(item, items=[]):
```

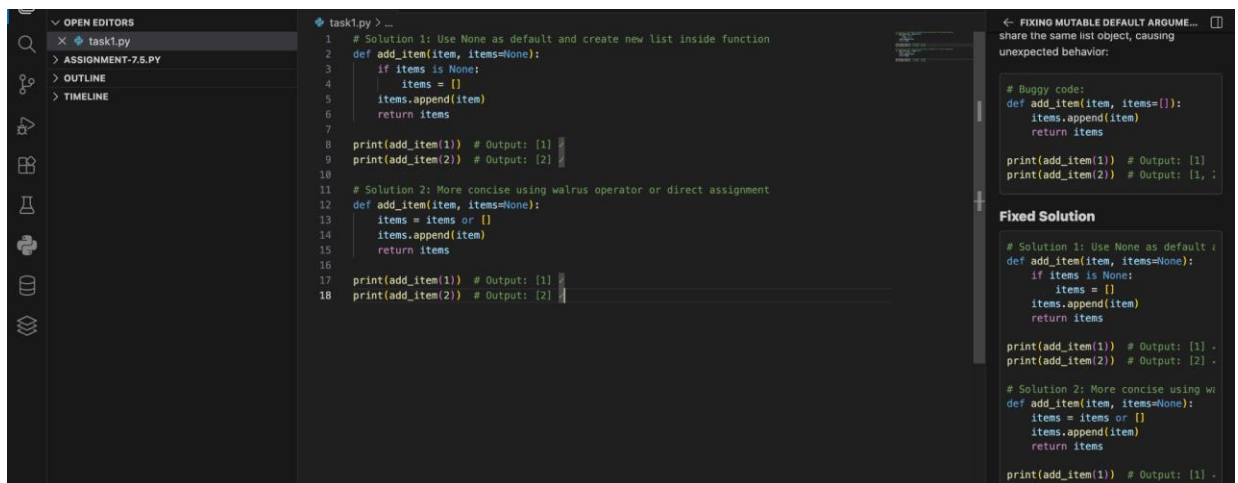
```
    items.append(item)
```

```
    return items
```

```
print(add_item(1))
```

```
print(add_item(2))
```

Expected Output: Corrected function avoids shared list bug.



The screenshot shows a code editor with a file named 'task1.py'. The main code area displays two solutions for the bug. Solution 1 uses 'None' as the default argument and creates a new list inside the function. Solution 2 uses the walrus operator or direct assignment to create a new list. The output of the code is shown in the console: 'Output: [1]' and 'Output: [2]'. A sidebar on the right shows the 'FIXING MUTABLE DEFAULT ARGUMENT...' section, which explains the bug and provides the fixed solutions.

```
1 # Solution 1: Use None as default and create new list inside function
2 def add_item(item, items=None):
3     if items is None:
4         items = []
5     items.append(item)
6     return items
7
8 print(add_item(1)) # Output: [1]
9 print(add_item(2)) # Output: [2]
10
11 # Solution 2: More concise using walrus operator or direct assignment
12 def add_item(item, items=None):
13     items = items or []
14     items.append(item)
15     return items
16
17 print(add_item(1)) # Output: [1]
18 print(add_item(2)) # Output: [2]
```

FIXING MUTABLE DEFAULT ARGUMENT...
share the same list object, causing unexpected behavior:

Buggy code:

```
def add_item(item, items=[]):
    items.append(item)
    return items

print(add_item(1)) # Output: [1]
print(add_item(2)) # Output: [1, 2]
```

Fixed Solution

Solution 1: Use None as default:

```
def add_item(item, items=None):
    if items is None:
        items = []
    items.append(item)
    return items

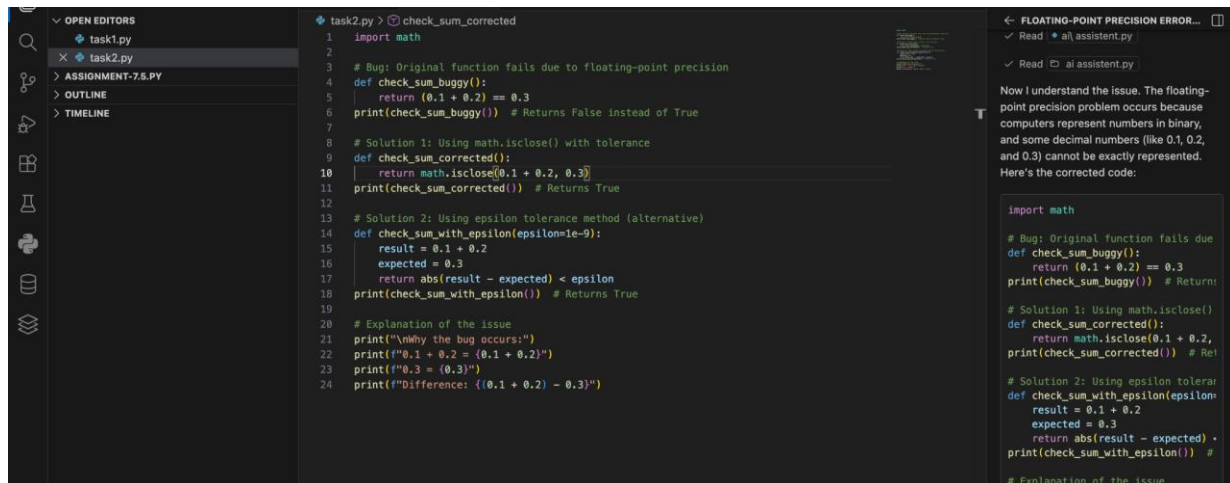
print(add_item(1)) # Output: [1]
print(add_item(2)) # Output: [2]
```

Solution 2: More concise using walrus operator:

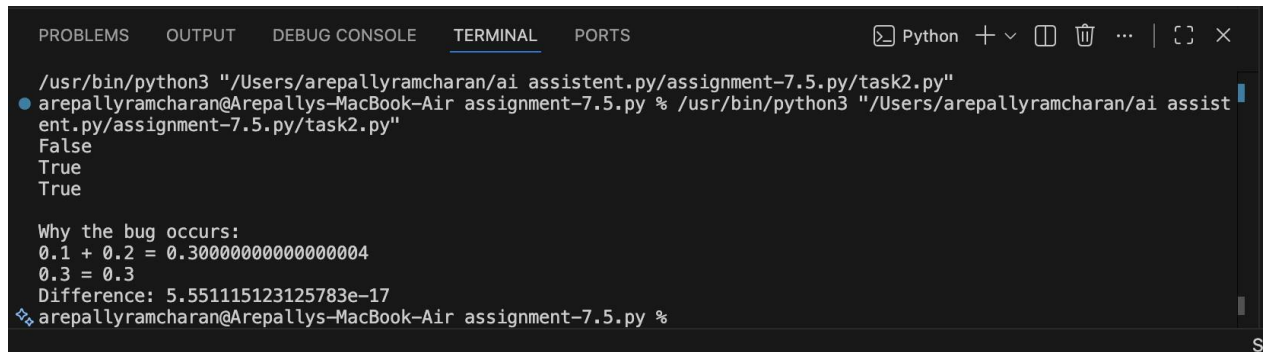
```
def add_item(item, items=None):
    items = items or []
    items.append(item)
    return items

print(add_item(1)) # Output: [1]
print(add_item(2)) # Output: [2]
```

Output:



Output:



Observation:

- Floating-point numbers are not always stored exactly in memory.
- Direct equality comparison (==) may fail due to small precision errors.
- Using a tolerance value avoids strict comparison issues.
- `math.isclose()` is the recommended safe comparison method.
- Proper floating-point handling improves numerical accuracy and reliability.

Task Description #3 (Transparency in Error Handling)

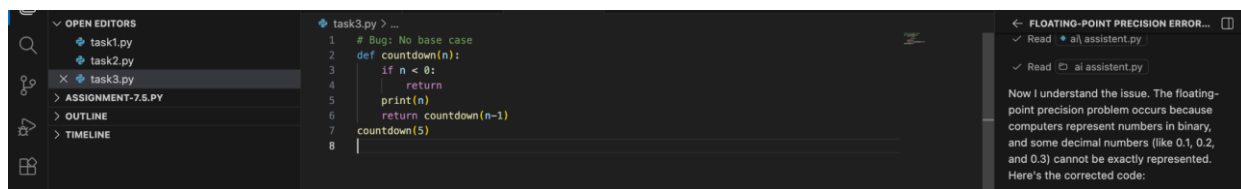
Task: Use AI to generate a Python program that reads a file and processes data.

Prompt:

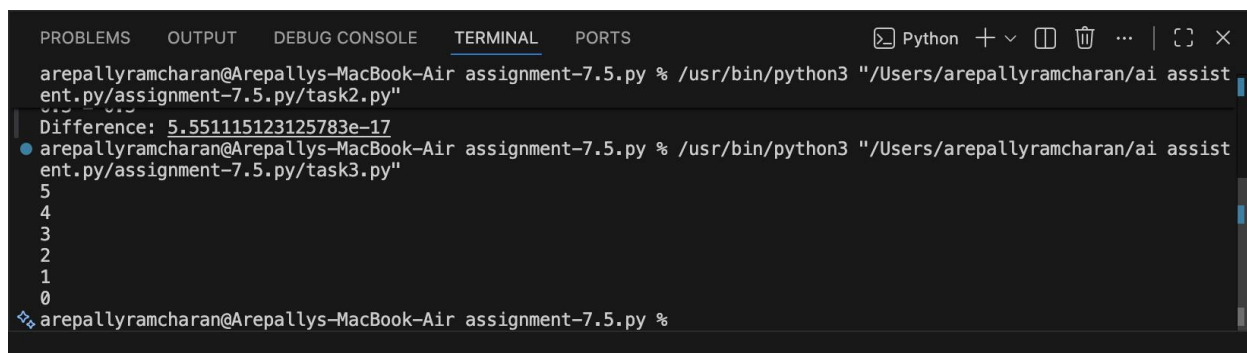
“Generate code with proper error handling and clear explanations for each exception.”

Expected Output:

- Code with meaningful exception handling.
- Clear comments explaining each error scenario.
- Validation that explanations align with runtime behavior.



Output:



Observation:

1. Recursion must always include a base case to stop execution.
2. Without a stopping condition, infinite recursion occurs.
3. Python limits recursion depth to prevent crashes.
4. Each recursive call should move toward the base case.
5. Proper base case design ensures predictable and safe recursion.

Task 4 (Dictionary Key Error)

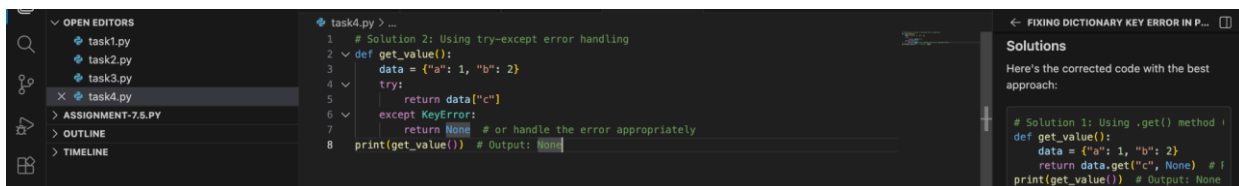
Task: Analyze given code where a missing dictionary key causes error. Use AI to fix it.

Bug: Accessing non-existing key

```
def get_value():  
    data = {"a": 1, "b": 2}  
    return data["c"]
```

```
print(get_value())
```

Expected Output: Corrected with `.get()` or error handling.



The screenshot shows a code editor with a file named `task4.py` open. The code defines a function `get_value()` that returns `data["c"]`, where `data` is a dictionary with keys `"a"` and `"b"`. The code is executed, and the output is `None`. The editor also shows a sidebar with a list of open editors and a timeline. On the right, a panel titled "Solutions" displays two AI-generated solutions for the error. Solution 1 uses the `.get()` method, and Solution 2 uses a try-except block to handle the `KeyError`.

```
1 # Solution 2: Using try-except error handling  
2 def get_value():  
3     data = {"a": 1, "b": 2}  
4     try:  
5         return data["c"]  
6     except KeyError:  
7         return None # or handle the error appropriately  
8 print(get_value()) # Output: None
```

Solutions
Here's the corrected code with the best approach:

```
# Solution 1: Using .get() method  
def get_value():  
    data = {"a": 1, "b": 2}  
    return data.get("c", None) # if  
print(get_value()) # Output: None
```

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + - [ ] [ ] ... [ ] [ ] X
/usr/bin/python3 "/Users/arepallyramcharan/ai assistant.py/assignment-7.5.py/task4.py"
arepallyramcharan@Arepallys-MacBook-Air assignment-7.5.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assist
ent.py/assignment-7.5.py/task4.py"
None
arepallyramcharan@Arepallys-MacBook-Air assignment-7.5.py %
```

Observation:

1. Direct dictionary access raises `KeyError` if the key does not exist.
2. `.get()` method provides safer access with a default value.
3. Exception handling prevents program crashes.
4. Defensive programming improves reliability.
5. Proper key validation ensures stable execution.

Task 5 (Infinite Loop – Wrong Condition)

Task: Analyze given code where loop never ends. Use AI to detect and fix it.

Bug: Infinite loop

```
def loop_example():
```

```
    i = 0
```

```
    while i < 5:
```

```
        print(i)
```

Expected Output: Corrected loop increments i.

```
task5.py > ...
1 # Bug: Infinite loop
2 def loop_example():
3     i = 0
4     while i < 5:
5         print(i)
6         i += 1
7
8 loop_example()
9
```

Solutions
Here's the corrected code with the best approach:

```
# Solution 1: Using .get() method
def get_value():
    data = {"a": 1, "b": 2}
    return data.get("c", None) # i
print(get_value()) # Output: None
```

Output:

```
arepallyramcharan@Arepallys-MacBook-Air assignment-7.5.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assistant.py/assignment-7.5.py/task5.py"
0
1
2
3
4
arepallyramcharan@Arepallys-MacBook-Air assignment-7.5.py %
```

Observation:

- Infinite loops occur when the loop variable never changes.
- The stopping condition must eventually become False.
- Loop control variables must be updated inside the loop.
- Careful condition checking prevents runtime hangs.
- Proper loop design ensures predictable execution flow.

Task 6 (Unpacking Error – Wrong Variables)

Task: Analyze given code where tuple unpacking fails. Use AI to fix it.

Bug: Wrong unpacking

a, b = (1, 2, 3)

Expected Output: Correct unpacking or using _ for extra values.

```
1 # Bug: Wrong unpacking - Fixed by adding third variable
2 a, b, c = (1, 2, 3)
3 print(a, b, c)
4
5 # Alternative: Use underscore to ignore extra values
6 a, b, _ = (1, 2, 3)
7 print(a, b)
8
```

Solutions

Here's the corrected code with the best approach:

```
# Solution 1: Using .get() method
def get_value():
    data = {'a': 1, 'b': 2}
    return data.get('c', None) # !
print(get_value()) # Output: None
```

Output:

```
/usr/bin/python3 "/Users/arepallyramcharan/ai assistant.py/assignment-7.5.py/task6.py"
arepallyramcharan@Arepallys-MacBook-Air assignment-7.5.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assistant.py/assignment-7.5.py/task6.py"
1 2 3
arepallyramcharan@Arepallys-MacBook-Air assignment-7.5.py %
```

Observation:

1. Number of variables must match number of elements in unpacking.
2. Mismatch causes `ValueError`.
3. `_` can be used to ignore unwanted values.
4. `*` operator allows flexible unpacking of remaining values.
5. Proper unpacking improves code readability and prevents runtime errors.

Task 7 (Mixed Indentation – Tabs vs Spaces)

Task: Analyze given code where mixed indentation breaks execution. Use AI to fix it.

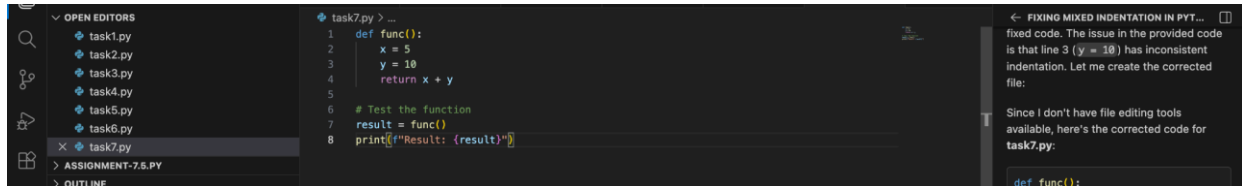
Bug: Mixed indentation

```
def func():
    x = 5
    y = 10
```



```
return x+y
```

Expected Output : Consistent indentation applied.

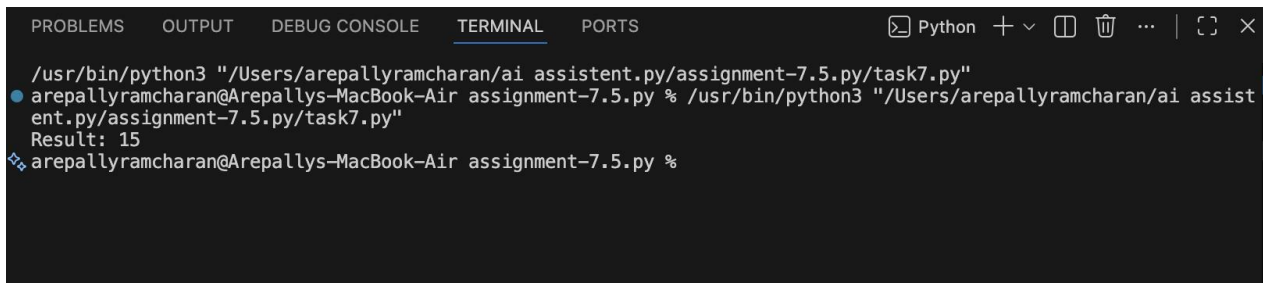


The screenshot shows a code editor with a file explorer on the left containing files task1.py through task6.py and task7.py. The main editor displays task7.py with the following code:

```
1 def func():
2     x = 5
3     y = 10
4     return x + y
5
6 # Test the function
7 result = func()
8 print(f"Result: {result}")
```

On the right, a panel titled "FIXING MIXED INDENTATION IN PYT..." contains text explaining an indentation issue in line 3 and offering a corrected version of the code.

Output:



The screenshot shows a terminal window with the following output:

```
/usr/bin/python3 "/Users/arepallyramcharan/ai assistant.py/assignment-7.5.py/task7.py"
arepallyramcharan@Arepallys-MacBook-Air assignment-7.5.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assistant.py/assignment-7.5.py/task7.py"
Result: 15
arepallyramcharan@Arepallys-MacBook-Air assignment-7.5.py %
```

Observation:

1. Python strictly enforces indentation rules.
2. Mixing tabs and spaces leads to `IndentationError` or `TabError`.
3. All statements inside a block must align consistently.
4. Standard convention is 4 spaces per indentation level.
5. Proper indentation improves readability and prevents syntax errors.

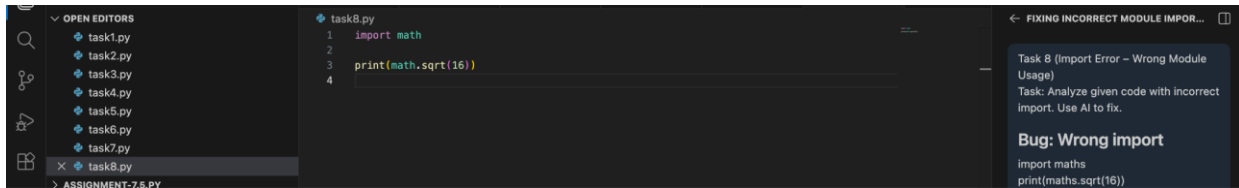
Task 8 (Import Error – Wrong Module Usage)

Task: Analyze given code with incorrect import. Use AI to fix.

Bug: Wrong import

```
import maths
print(maths.sqrt(16))
```

Expected Output: Corrected to import math



Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
Python + v [icon] [icon] [icon] [icon] [icon] [icon] [icon] [icon]

/usr/bin/python3 "/Users/arepallyramcharan/ai assistant.py/assignment-7.5.py/task8.py"
arepallyramcharan@Arepallys-MacBook-Air assignment-7.5.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assist
ent.py/assignment-7.5.py/task8.py"
4.0
arepallyramcharan@Arepallys-MacBook-Air assignment-7.5.py %
```

Observation:

1. Python module names must be spelled correctly.
2. Incorrect module names cause `ModuleNotFoundError`.
3. Python module names are case-sensitive.
4. Always verify module documentation before importing.
5. Correct imports ensure proper access to built-in functions.

Task 9 (Unreachable Code – Return Inside Loop)

Task: Analyze given code where a return inside a loop prevents full iteration. Use AI to fix it.

Bug: Early return inside loop

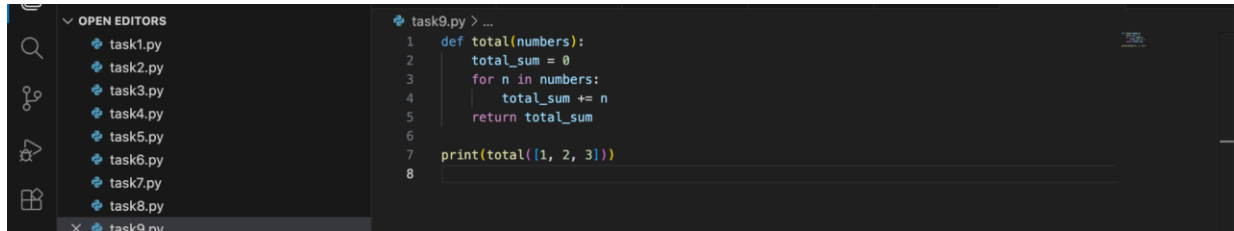
```
def total(numbers):
```

```
    for n in numbers:
```

```
        return n
```

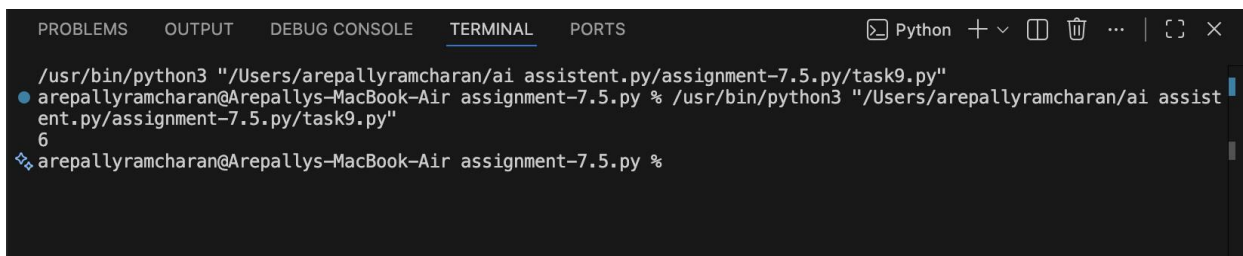
```
print(total([1,2,3]))
```

Expected Output: Corrected code accumulates sum and returns after loop.



```
task9.py > ...
1 def total(numbers):
2     total_sum = 0
3     for n in numbers:
4         total_sum += n
5     return total_sum
6
7 print(total([1, 2, 3]))
8
```

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + v [ ] [ ] ... [ ] [ ] [ ] [ ]
/usr/bin/python3 "/Users/arepallyramcharan/ai assistant.py/assignment-7.5.py/task9.py"
● arepallyramcharan@Arepallys-MacBook-Air assignment-7.5.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assist
ent.py/assignment-7.5.py/task9.py"
6
❖ arepallyramcharan@Arepallys-MacBook-Air assignment-7.5.py %
```

Observation:

1. A `return` statement immediately exits the function.
2. Placing `return` inside a loop may stop full iteration.
3. Code after early return becomes unreachable.
4. Accumulation logic should complete before returning result.
5. Proper return placement ensures correct computation.

Task 10 (Name Error – Undefined Variable)

Task: Analyze given code where a variable is used before being defined. Let AI detect and fix the error.

Bug: Using undefined variable

```
def calculate_area():  
    return length * width  
print(calculate_area())
```

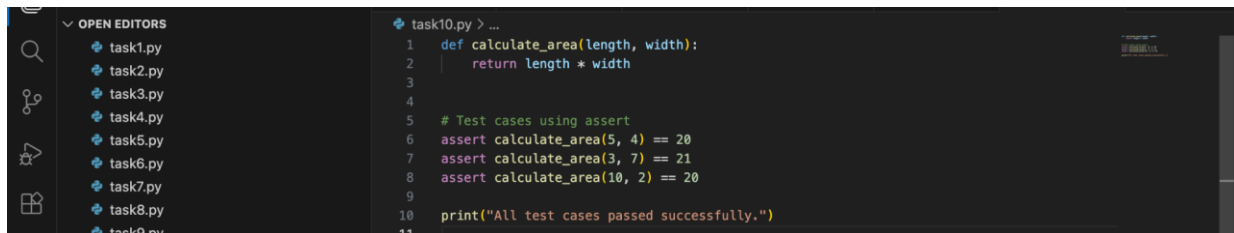
Requirements:

- Run the code to observe the error.
- Ask AI to identify the missing variable definition.
- Fix the bug by defining length and width as parameters.
- Add 3 assert test cases for correctness.

Expected Output :

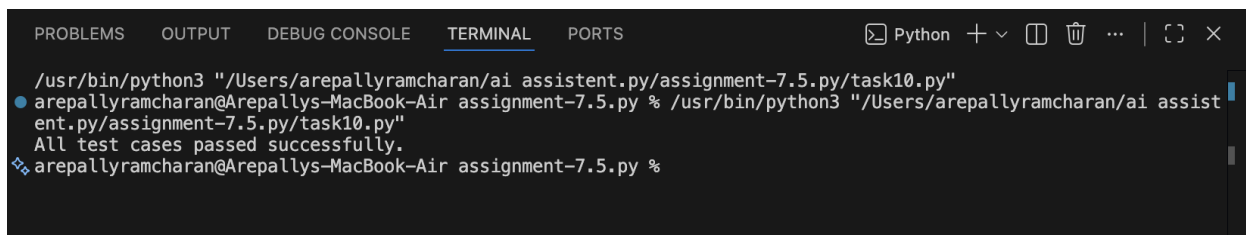
- Corrected code with parameters.
- AI explanation of the bug.

Successful execution of assertions.



```
task10.py > ...  
1  def calculate_area(length, width):  
2      return length * width  
3  
4  
5  # Test cases using assert  
6  assert calculate_area(5, 4) == 20  
7  assert calculate_area(3, 7) == 21  
8  assert calculate_area(10, 2) == 20  
9  
10 print("All test cases passed successfully.")  
11
```

Output:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
Python + v [icon] [icon] ... [icon] [icon] [icon]  
/usr/bin/python3 "/Users/arepallyramcharan/ai assistant.py/assignment-7.5.py/task10.py"  
● arepallyramcharan@Arepallys-MacBook-Air assignment-7.5.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assist  
ent.py/assignment-7.5.py/task10.py"  
All test cases passed successfully.  
arepallyramcharan@Arepallys-MacBook-Air assignment-7.5.py %
```

Observation:

1. The function attempted to use variables that were not defined.

2. Variables inside a function must either:
 - Be defined inside the function, or
 - Be passed as parameters.
3. Since `length` and `width` were missing, Python raised a `NameError`.
4. Adding them as function parameters fixes the scope issue.
5. Assertions confirm correctness of the function logic.

Task 11 (Type Error – Mixing Data Types Incorrectly)

Task: Analyze given code where integers and strings are added incorrectly. Let AI detect and fix the error.

Bug: Adding integer and string

```
def add_values():  
    return 5 + "10"  
print(add_values())
```

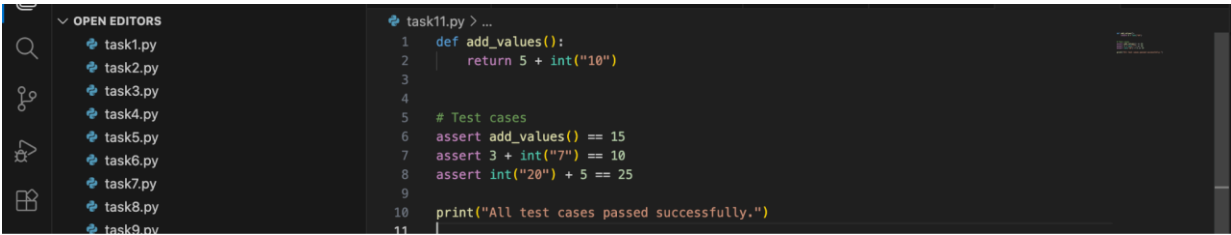
Requirements:

- Run the code to observe the error.
- AI should explain why `int + str` is invalid.
- Fix the code by type conversion (e.g., `int("10")` or `str(5)`).
- Verify with 3 assert cases.

Expected Output #6:

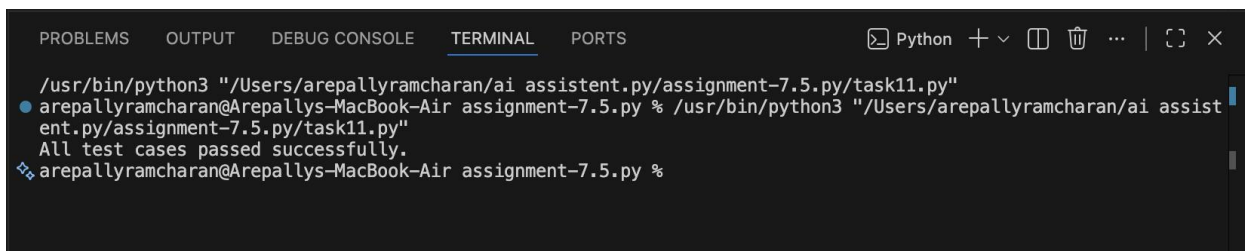
- Corrected code with type handling.
- AI explanation of the fix.

Successful test validation.



```
task11.py > ...
1  def add_values():
2      return 5 + int("10")
3
4
5  # Test cases
6  assert add_values() == 15
7  assert 3 + int("7") == 10
8  assert int("20") + 5 == 25
9
10 print("All test cases passed successfully.")
11
```

Output:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
Python + - [ ] [ ] ... | [ ] [ ] x

/usr/bin/python3 "/Users/arepallyramcharan/ai assistant.py/assignment-7.5.py/task11.py"
arepallyramcharan@Arepallys-MacBook-Air assignment-7.5.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assistant.py/assignment-7.5.py/task11.py"
All test cases passed successfully.
arepallyramcharan@Arepallys-MacBook-Air assignment-7.5.py %
```

Observation:

1. The error occurs because Python is strongly typed.
2. Integers and strings cannot be added directly.
3. Explicit type conversion is required.
4. Converting "10" to `int` enables numeric addition.
5. Assertions confirm the corrected logic works properly.

Task 12 (Type Error – String + List Concatenation)

Task: Analyze code where a string is incorrectly added to a list.

Bug: Adding string and list

```
def combine():  
    return "Numbers: " + [1, 2, 3]  
print(combine())
```

Requirements:

- Run the code to observe the error.
- Explain why str + list is invalid.
- Fix using conversion (str([1,2,3]) or " ".join()).
- Verify with 3 assert cases.

Expected Output:

- Corrected code
- Explanation
- Successful test validation

```
task12.py > ...  
1 def combine():  
2     return "Numbers: " + str([1, 2, 3])  
3  
4  
5 # Test cases  
6 assert combine() == "Numbers: [1, 2, 3]"  
7 assert "Numbers: " + str([4, 5]) == "Numbers: [4, 5]"  
8 assert str([7]) == "[7]"  
9  
10 print("All test cases passed successfully.")  
11
```

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS  
Python + - [ ] [ ] ... [ ] [ ] [ ]  
/usr/bin/python3 "/Users/arepallyramcharan/ai assistant.py/assignment-7.5.py/task12.py"  
arepallyramcharan@Arepallys-MacBook-Air assignment-7.5.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assist  
ent.py/assignment-7.5.py/task12.py"  
All test cases passed successfully.  
arepallyramcharan@Arepallys-MacBook-Air assignment-7.5.py %
```

Observation:

1. Python is strongly typed and does not allow `str + list`.
2. Both operands must be of the same type for concatenation.
3. `str([1,2,3])` converts the list to a string representation.
4. `" ".join(map(str, list))` converts each element to string and joins them cleanly.
5. Assertions confirm the corrected function works properly.

Task 13 (Type Error – Multiplying String by Float)

Task: Detect and fix code where a string is multiplied by a float.

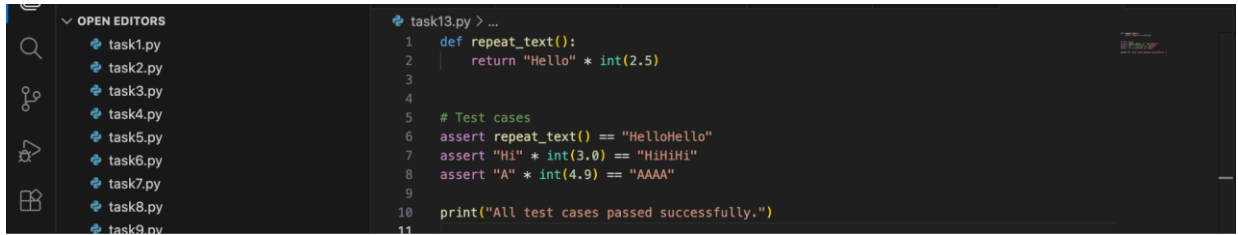
Bug: Multiplying string by float

```
def repeat_text():  
    return "Hello" * 2.5
```

```
print(repeat_text())
```

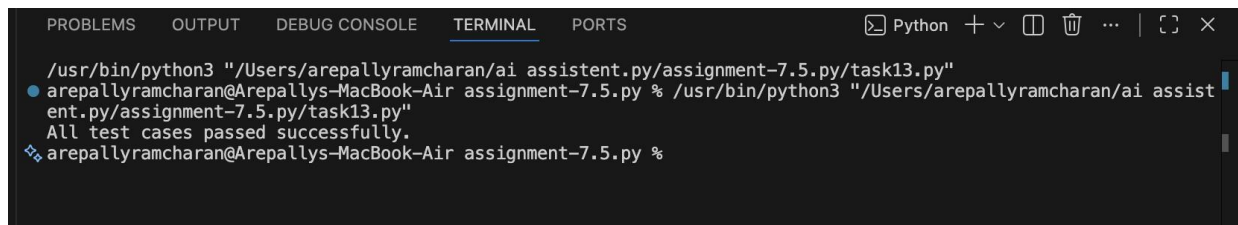
Requirements:

- Observe the error.
- Explain why float multiplication is invalid for strings.
- Fix by converting float to int.
- Add 3 assert test cases.



```
task13.py > ...
1  def repeat_text():
2      return "Hello" * int(2.5)
3
4
5  # Test cases
6  assert repeat_text() == "HelloHello"
7  assert "Hi" * int(3.0) == "HiHiHi"
8  assert "A" * int(4.0) == "AAAA"
9
10 print("All test cases passed successfully.")
11
```

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + - [ ] [x] [ ] [x]
/usr/bin/python3 "/Users/arepallyramcharan/ai assistant.py/assignment-7.5.py/task13.py"
● arepallyramcharan@Arepallys-MacBook-Air assignment-7.5.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assist
ent.py/assignment-7.5.py/task13.py"
All test cases passed successfully.
❖ arepallyramcharan@Arepallys-MacBook-Air assignment-7.5.py %
```

Observation:

String multiplication requires an integer.

Float values must be converted using `int()`.

`int(2.5)` becomes `2` (truncates decimal).

After conversion, repetition works correctly.

Assertions verify correct functionality.

Task 14 (Type Error – Adding None to Integer)

Task: Analyze code where None is added to an integer.

Bug: Adding None and integer

def compute():

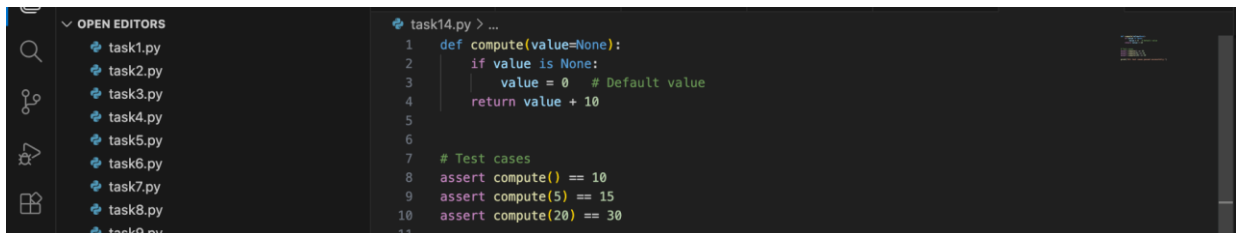
```
value = None
```

```
return value + 10
```

```
print(compute())
```

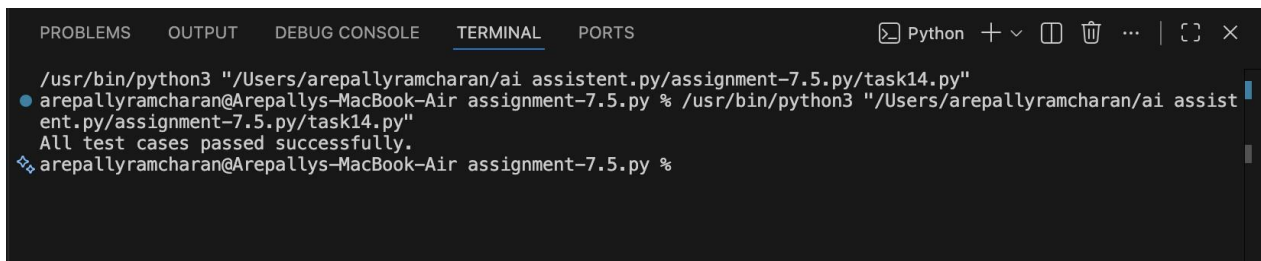
Requirements:

- Run and identify the error.
- Explain why NoneType cannot be added.
- Fix by assigning a default value.
- Validate using asserts.



```
task14.py > ...
1  def compute(value=None):
2      if value is None:
3          value = 0 # Default value
4          return value + 10
5
6
7  # Test cases
8  assert compute() == 10
9  assert compute(5) == 15
10 assert compute(20) == 30
11
```

Output:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
Python + - [ ] [ ] ... | [ ] [ ] X

/usr/bin/python3 "/Users/arepallyramcharan/ai assistent.py/assignment-7.5.py/task14.py"
● arepallyramcharan@Arepallys-MacBook-Air assignment-7.5.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assistent.py/assignment-7.5.py/task14.py"
All test cases passed successfully.
arepallyramcharan@Arepallys-MacBook-Air assignment-7.5.py %
```

Observation:

`None` cannot participate in arithmetic operations.

We check if `value is None`.

If so, assign a safe default numeric value (`0`).

This prevents `TypeError`.

Assertions confirm correct behavior.

Task 15 (Type Error – Input Treated as String Instead of Number)

Task: Fix code where user input is not converted properly.

Bug: Input remains string

```
def sum_two_numbers():
```

```
    a = input("Enter first number: ")
```

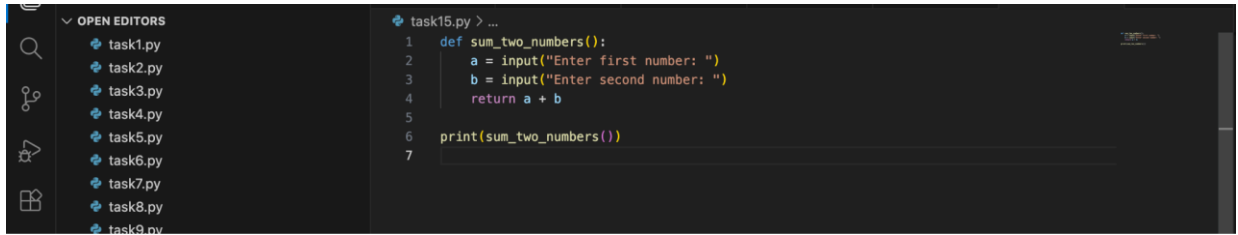
```
    b = input("Enter second number: ")
```

```
    return a + b
```

```
print(sum_two_numbers())
```

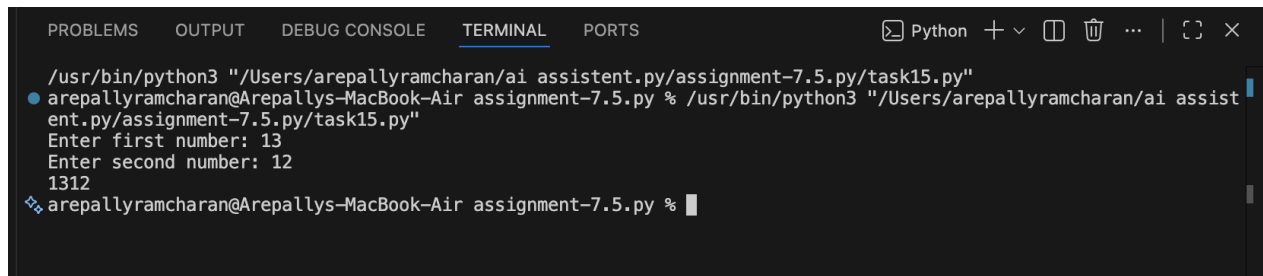
Requirements:

- Explain why input is always string.
- Fix using `int()` conversion.
- Verify with assert test cases.



```
task15.py > ...
1 def sum_two_numbers():
2     a = input("Enter first number: ")
3     b = input("Enter second number: ")
4     return a + b
5
6 print(sum_two_numbers())
7
```

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + - [ ] [ ] ... [ ] [ ] [ ] [ ]
/usr/bin/python3 "/Users/arepallyramcharan/ai assistant.py/assignment-7.5.py/task15.py"
● arepallyramcharan@Arepallys-MacBook-Air assignment-7.5.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assist
ent.py/assignment-7.5.py/task15.py"
Enter first number: 13
Enter second number: 12
1312
❖ arepallyramcharan@Arepallys-MacBook-Air assignment-7.5.py %
```

Observation:

`input()` always returns string data.

String addition results in concatenation.

Use `int()` to convert string to integer.

After conversion, `+` performs numeric addition.

Assertions confirm correct functionality.